

## Introduction to Feed-Forward Neural Network in Deep Learning

Home > Beginner

> Introduction to Feed-Forward Neural Network in Deep Learning

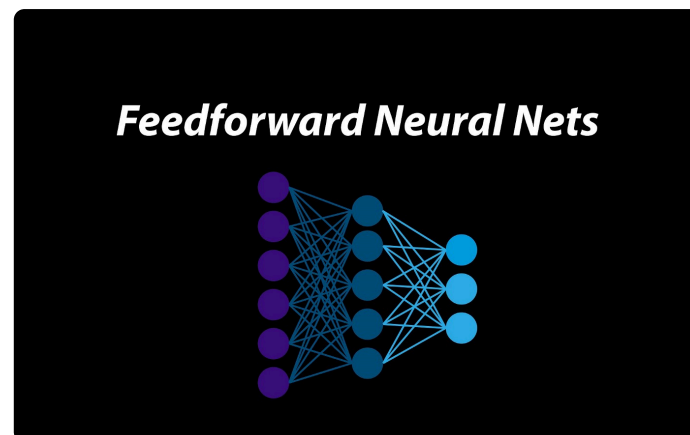


[Pranshu Sharma](#)

08 Feb, 2024 • 7 min read

This article was published as a part of the [Data Science Blogathon](#).

***A Feed Forward Neural Network is an artificial Neural Network in which the nodes are connected circularly.*** A feed-forward neural network, in which some routes are cycled, is the polar opposite of a Recurrent Neural Network. The feed-forward model is the basic type of neural network because the input is only processed in one direction. The data always flows in one direction and never backwards/opposite.



Source: Medium

***The Neural Network advanced from the perceptron, a prominent machine learning algorithm. Frank Rosenblatt, a physicist, invented perceptrons in the 1950s and 1960s, based on earlier work by Warren McCulloch and Walter Pitts.***

[Table of contents](#)

## Why are Neural Networks used?

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

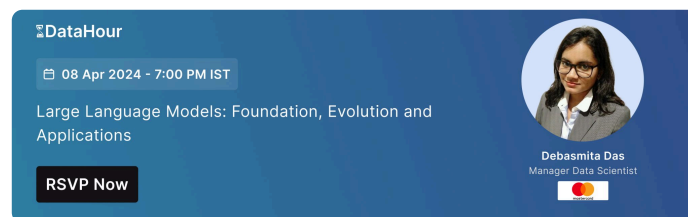
## Introduction to Feed-Forward Neural Network in Deep Learning

*inputs with outputs. In theory, neural networks should be able to estimate any sort of function, no matter how complex it is.* Nonetheless, supervised learning entails learning a function that translates a given  $X$  to a specified  $Y$  and then utilising that function to determine the proper  $Y$  for a fresh  $X$ . If that's the case, how do [neural networks](#) differ from typical machine learning methods? Inductive Bias, a psychological phenomenon, is the answer. The phrase may appear to be fresh. However, before applying a machine learning model to it, it is nothing more than our



Bias of linear regression. As a result, it fits the data to a line or a hyperplane.

*When there is a non-linear and complex relationship between  $X$  and  $Y$ , nevertheless, a Linear Regression method may struggle to predict  $Y$ .* To approximate that relationship, we may need a curve or a multi-dimensional curve in this scenario.



However, depending on the function's complexity, we may need to manually set the number of neurons in each layer and the total number of layers in the network. This is usually accomplished through trial and error methods as well as experience. As a result, these parameters are referred to as hyperparameters.

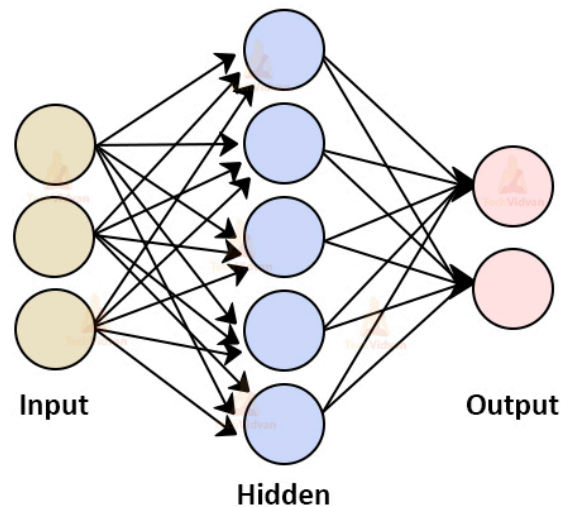
## Neural Network Architecture and Operation

*Before we look at why neural networks work, it's important to understand what neural networks do. Before we can grasp the design of a neural network, we must first understand what a neuron performs.*

## Introduction to Feed-Forward Neural Network in Deep Learning

sum is passed via an activation function, being a non-linear function.

### Architecture of Artificial Neural Network



Source: TowardsDatascience

***A weight is being applied to each input to an artificial neuron. First, the inputs are multiplied by their weights, and then a bias is applied to the outcome.*** This is called the weighted sum. After that, the weighted sum is processed via an activation function, as a non-linear function.

The first layer is the input layer, which appears to have six neurons but is only the data that is sent into the [neural network](#). The output layer is the final layer. The dataset and the type of challenge determine the number of neurons in the final layer and the first layer. Trial and error will be used to determine the number of neurons in the hidden layers and the number of hidden layers.

All of the inputs from the previous layer will be connected to the first neuron from the first hidden layer. The second neuron in the first hidden layer will be connected to all of the preceding layer's inputs, and so forth for all of the first hidden layer's neurons. The outputs of the previously hidden layer are regarded inputs for neurons in the second hidden layer, and each of these neurons is

## Introduction to Feed-Forward Neural Network in Deep Learning

***In its most basic form, a Feed-Forward [Neural Network](#) is a single layer perceptron. A sequence of inputs enter the layer and are multiplied by the weights in this model.***

***The weighted input values are then summed together to***

***form a total.*** If the sum of the values is more than a

predetermined threshold, which is normally set at zero,

the output value is usually 1, and if the sum is less than

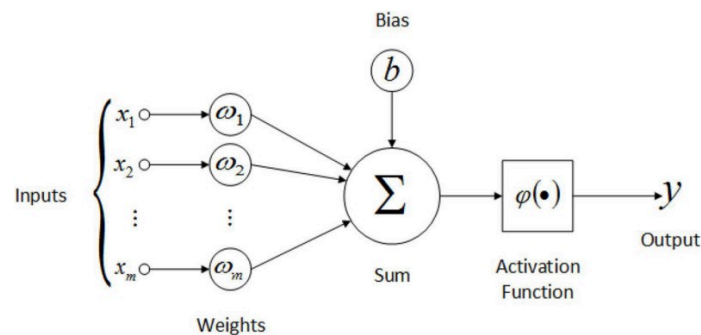
the threshold, the output value is usually -1. The single-

layer perceptron is a popular [feed-forward neural network](#)

model that is frequently used for classification. Single-

layer perceptrons can also contain machine learning

features.



Source: Medium.com

***The neural network can compare the outputs of its nodes with the desired values using a property known as the delta rule, allowing the network to alter its weights through training to create more accurate output***

***values.*** This training and learning procedure results in

gradient descent. The technique of updating weights in

multi-layered perceptrons is virtually the same, however,

the process is referred to as back-propagation. In such

circumstances, the output values provided by the final

layer are used to alter each hidden layer inside the

[network](#).

## Python Implementation

**Representing the [feed-forward neural network](#) using Python**

## Introduction to Feed-Forward Neural Network in Deep Learning

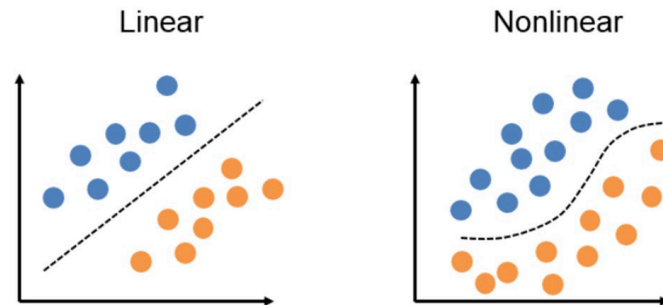
```
import numpy as np
from sklearn import datasets
#
# Generate a dataset and plot it
#
np.random.seed(0)
X, y = datasets.make_moons(200, noise=0.20)
#
# Neural network architecture
# No of nodes in input layer = 4
# No of nodes in output layer = 3
# No of nodes in the hidden layer = 6
#
input_dim = 4 # input layer dimensionality
output_dim = 3 # output layer dimensionality
hidden_dim = 6 # hidden layer dimensionality
#
# Weights and bias element for layer 1
# These weights are applied for calculating
# weighted sum arriving at neurons in 1st hidden layer
#
W1 = np.random.randn(input_dim, hidden_dim)
b1 = np.zeros((1, hidden_dim))
#
# Weights and bias element for layer 2
# These weights are applied for calculating
# weighted sum arriving at neurons in 2nd hidden layer
#
W2 = np.random.randn(hidden_dim, hidden_dim)
b2 = np.zeros((1, hidden_dim))
#
# Weights and bias element for layer 2
# These weights are applied for calculating
# weighted sum arriving at in the final / output layer
#
W3 = np.random.randn(hidden_dim, output_dim)
b3 = np.zeros((1, output_dim))
```

***Python code implementation for the propagation of the input signal through different layers towards the output layer***

```
#
# Forward propagation of input signals
# to 6 neurons in first hidden layer
# activation is calculated based tanh function
#
z1 = X.dot(W1) + b1
a1 = np.tanh(z1)
#
# Forward propagation of activation signals from first hidden layer
# to 6 neurons in second hidden layer
# activation is calculated based tanh function
#
z2 = a1.dot(W2) + b2
a2 = np.tanh(z2)
#
# Forward propagation of activation signals from second hidden layer
# to 3 neurons in output layer
#
z3 = a2.dot(W3) + b3
#
# Probability is calculated as an output
# of softmax function
"
```

## Introduction to Feed-Forward Neural Network in Deep Learning

***As we've seen, the function of each neurone in the network is similar to that of linear regression. The neuron also has an activation function at the end, and each neuron has its weight vector.***



Source: Medium.com

We've seen how the computation works so far. But the major purpose of this blog is to explain why this strategy works. [Neural networks](#) should theoretically be able to estimate any continuous function, no matter how complex or non-linear it is.

### Importance of the Non-Linearity

***When two or more linear objects, such as a line, plane, or hyperplane, are combined, the outcome is also a linear object: line, plane, or hyperplane.*** No matter how many of these linear things we add, we'll still end up with a linear object.

However, this is not the case when adding non-linear objects. When two separate curves are combined, the result is likely to be a more complex curve.

***We're introducing non-linearity at every layer using these activation functions, in addition to just adding non-linear objects or hyper-curves like hyperplanes.*** In other words, we're applying a nonlinear function on an already nonlinear object.

What if activation functions were not used in [neural networks](#)?

## Introduction to Feed-Forward Neural Network in Deep Learning

$$a = m \cdot x + d$$

$$\begin{aligned} Z &= k \cdot a + t \Rightarrow k \cdot (m \cdot x + d) + t \Rightarrow k \cdot m \cdot x + k \cdot d + t \Rightarrow \\ &(k \cdot m) \cdot x + (k \cdot d + t) \end{aligned}$$

## Applications of the Feed Forward Neural Networks

***A Feed Forward Neural Network is an artificial neural network in which the nodes are connected circularly. A feed-forward neural network, in which some routes are cycled, is the polar opposite of a recurrent neural network.*** The feed-forward [model](#) is the simplest type of neural network because the input is only processed in one direction. The data always flows in one direction and never backwards, regardless of how many buried nodes it passes through.

## Conclusion

In this article, we covered the basic Introduction to [Feed-Forward](#) Neural Networks and their basic implementation in Python. We also covered their workings, architecture, and applications in real-world scenarios. Read more articles on our [blog](#).

Hope you liked my article on the feed-forward network?  
Please share in the comments below.

**The media shown in this article is not owned by Analytics Vidhya and are used at the Author's discretion.**

[blogathon](#)[feed forward network](#)[Feed Forward neural Network](#)[feed forward neural network example](#)[what is feed forward neural network](#)

## Introduction to Feed-Forward Neural Network in Deep Learning

---

### Frequently Asked Questions

Q1. What is feed-forward vs deep feed-forward?

A. Feed-forward refers to a neural network architecture where information flows in one direction, from input to output, with no feedback loops. Deep feed-forward, commonly known as a deep neural network, consists of multiple hidden layers between input and output layers, enabling the network to learn complex hierarchical features and patterns, enhancing its ability to model intricate relationships in data.

Q2. What is feed-forward vs feedback neural network?



## Introduction to Feed-Forward Neural Network in Deep Learning

What are your thoughts?...

Submit reply



M. Sudhakara

17 Jul, 2023

A Feed Forward Neural Network is an artificial neural network in which the nodes are connected circularly



## Related Courses



• 21 Lessons ★ 4.9

Introduction to  
Neural Networks

**FREE**

Deep Learning

Neural Networks

Enroll now



• 28 Lessons ★ 4.8

Introduction to  
PyTorch for Deep  
Learning

**FREE**

Deep Learning

Neural Networks

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

## Introduction to Feed-Forward Neural Network in Deep Learning



• 72 Lessons ★ 4.84

### A Comprehensive Learning Path for Deep Learning in 2023

**FREE**

Learning Path

Deep Learning

Neural Networks

[Enroll now](#)

## Write, Shine, Succeed →

Write, captivate, and earn accolades and rewards for your work

- ✓ Reach a Global Audience
- ✓ Get Expert Feedback
- ✓ Build Your Brand & Audience
- ✓ Cash In on Your Knowledge
- ✓ Join a Thriving Community
- ✓ Level Up Your Data Science Game



Sion Chakrabarti  
16



CHIRAG GOJ  
87

Company

Discover

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

Directo                  Indirecto

## Engage

Community

## Hackathons

## Events

## Daily challenges

## Enterprise

## Our offerings

## Case studies

## Industry report

quexto.ai



[Terms & conditions](#) • [Refund Policy](#) • [Privacy Policy](#) • [Cookies Policy](#) © Analytics Vidhya 2023.All rights reserved.

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#).